

Automation & QA Developer Assessment

Task 2 & Bonus — n8n Workflow Submission

Srilatha Pothana

May 2026

Section 1 — Task 2 README

Workflow name: HackerNews Morning Brief

File: Task2_Workflow_SrilathaPothana.json

APIs Used & Why

API	Endpoint	Why
HackerNews Firebase API	/v0/topstories.json /v0/item/{id}.json	Free, no auth needed. Returns live top story IDs and full story detail (title, score, author, comments).
Algolia HN Search API	/api/v1/search?tags=story&query=...	Free, no auth. Enriches high-score stories with additional metadata and ranking signals.
Discord Webhook	Incoming webhook URL	Free, instant Markdown delivery — ideal for team morning digests.

Workflow Flow

Schedule Trigger (every 1 hour)

→ HN Firebase: fetch top 500 story IDs

→ Code node: slice to first 5 IDs

→ HN Firebase: fetch full details for each story (runs 5×)

→ IF node: score > 100?

YES → Algolia: enrich with extra metadata → Merge node → Build digest

NO → Label as NORMAL → Build digest

→ Discord webhook: post formatted digest

Transformation Logic

The **Code node "Transform – Top 5 IDs"** slices the HackerNews array (up to 500 IDs) to just the first 5, then maps each into an item object for parallel fetching. The **"Build – Discord Digest Message"** node aggregates results from both IF branches into a single Markdown-formatted message, using **■** for high-score stories and **■**

for normal ones.

IF Branch — Conditional Logic

- **Threshold:** score > 100
- **TRUE (High score):** Enriched via Algolia API for additional engagement metadata.
- **FALSE (Normal score):** Passed through as-is with a NORMAL tier label — still included in digest.
- **Rationale:** Algolia calls add latency; limiting enrichment to trending stories keeps execution fast.

Error Handling

Every HTTP Request node has **onError: continueErrorOutput** configured. Failed calls are routed to the **Error Handler – Log & Continue** Code node, which logs a structured entry with timestamp and node name. The workflow never crashes silently. In production this node would write to a Google Sheet or trigger a fallback email alert.

Credentials Setup

Secrets are never hardcoded. In n8n:

- Go to **Settings** → **Credentials** → **New**
- Create a credential of type **HTTP Custom Auth** named *Discord Webhook*
- Set the webhook URL as a credential field
- Reference it in nodes via `$credentials.discordWebhookUrl`

How to Import

- Open your n8n instance
- Click **Workflows** → **Import from File**
- Select **Task2_Workflow_SrilathaPothana.json** (included in Section 2 below)
- Create the Discord Webhook credential as described above
- Activate the workflow — it fires every hour automatically

Section 2 — Task 2 Workflow JSON

(Task2_Workflow_SrilathaPothana.json)

Import this JSON directly into n8n via Workflows → Import from File.

```
{
  "name": "Task2 \u2013 HackerNews Morning Brief",
  "nodes": [
    {
      "parameters": {
        "rule": {
          "interval": [
            {
              "field": "hours",
              "hoursInterval": 1
            }
          ]
        }
      },
      "id": "node-schedule",
      "name": "Schedule Trigger (Every 1h)",
      "type": "n8n-nodes-base.scheduleTrigger",
      "typeVersion": 1.2,
      "position": [
        0,
        300
      ]
    },
    {
      "parameters": {
        "url": "https://hacker-news.firebaseio.com/v0/topstories.json",
        "options": {}
      },
      "id": "node-hn-top",
      "name": "HN \u2013 Get Top Story IDs",
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.2,
      "position": [
        220,
        300
      ],
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "jsCode": "// Grab first 5 story IDs from the array\nconst ids = $input.first().json.slice(0, 5);\nreturn ids.map"
      },
      "id": "node-transform",
      "name": "Transform \u2013 Top 5 IDs",
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        440,
        300
      ]
    },
    {
      "parameters": {
        "url": "https://hacker-news.firebaseio.com/v0/item/{{ $json.id }}.json",
```

```

    "options": {}
  },
  "id": "node-hn-story",
  "name": "HN \u2013 Fetch Each Story",
  "type": "n8n-nodes-base.httpRequest",
  "typeVersion": 4.2,
  "position": [
    660,
    300
  ],
  "onError": "continueErrorOutput"
},
{
  "parameters": {
    "conditions": {
      "options": {
        "caseSensitive": true,
        "leftValue": "",
        "typeValidation": "strict"
      },
      "conditions": [
        {
          "id": "cond-score",
          "leftValue": "={{ $json.score }}",
          "rightValue": 100,
          "operator": {
            "type": "number",
            "operation": "gt"
          }
        }
      ]
    },
    "combinator": "and"
  },
  "options": {}
},
  "id": "node-if",
  "name": "IF \u2013 Score > 100",
  "type": "n8n-nodes-base.if",
  "typeVersion": 2,
  "position": [
    880,
    300
  ]
},
{
  "parameters": {
    "url": "https://hn.algolia.com/api/v1/search?tags=story&query={{ encodeURIComponent($json.title) }}&hitsPerPage=1",
    "options": {}
  },
  "id": "node-enrich-high",
  "name": "Enrich \u2013 Algolia Details (High Score)",
  "type": "n8n-nodes-base.httpRequest",
  "typeVersion": 4.2,
  "position": [
    1100,
    160
  ],
  "onError": "continueErrorOutput"
},
{
  "parameters": {

```

```

    "jsCode": "// Combine original story data with algolia enrichment\nconst story = $('IF \u2013 Score > 100').first()
  },
  "id": "node-merge-high",
  "name": "Merge \u2013 High Score Story",
  "type": "n8n-nodes-base.code",
  "typeVersion": 2,
  "position": [
    1320,
    160
  ]
},
{
  "parameters": {
    "jsCode": "// Low-score stories: pass through with tier label\nconst story = $input.first().json;\nreturn [{\n  ]
  },
  "id": "node-merge-low",
  "name": "Label \u2013 Normal Score Story",
  "type": "n8n-nodes-base.code",
  "typeVersion": 2,
  "position": [
    1100,
    440
  ]
},
{
  "parameters": {
    "jsCode": "// Aggregate all items from both branches into a single Discord message\nconst allItems = $input.all()
  },
  "id": "node-build-digest",
  "name": "Build \u2013 Discord Digest Message",
  "type": "n8n-nodes-base.code",
  "typeVersion": 2,
  "position": [
    1540,
    300
  ]
},
{
  "parameters": {
    "method": "POST",
    "url": "={{ $credentials.discordWebhookUrl }}",
    "sendBody": true,
    "contentType": "json",
    "body": {
      "content": "={{ $json.message }}"
    },
    "options": {}
  },
  "id": "node-discord",
  "name": "Discord \u2013 Post Digest",
  "type": "n8n-nodes-base.httpRequest",
  "typeVersion": 4.2,
  "position": [
    1760,
    300
  ],
  "credentials": {
    "httpCustomAuth": {
      "id": "cred-discord",
      "name": "Discord Webhook"
    }
  }
}

```

```

    },
    "onError": "continueErrorOutput"
  },
  {
    "parameters": {
      "jsCode": "// Error handler: log details about which node failed\nconst err = $input.first();\nconst ts = new Dat
    },
    "id": "node-error-handler",
    "name": "Error Handler \u2013 Log & Continue",
    "type": "n8n-nodes-base.code",
    "typeVersion": 2,
    "position": [
      880,
      560
    ]
  }
],
"connections": {
  "Schedule Trigger (Every 1h)": {
    "main": [
      [
        {
          "node": "HN \u2013 Get Top Story IDs",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "HN \u2013 Get Top Story IDs": {
    "main": [
      [
        {
          "node": "Transform \u2013 Top 5 IDs",
          "type": "main",
          "index": 0
        }
      ],
      [
        {
          "node": "Error Handler \u2013 Log & Continue",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Transform \u2013 Top 5 IDs": {
    "main": [
      [
        {
          "node": "HN \u2013 Fetch Each Story",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "HN \u2013 Fetch Each Story": {
    "main": [
      [

```

```

    {
      "node": "IF \u2013 Score > 100",
      "type": "main",
      "index": 0
    }
  ],
  [
    {
      "node": "Error Handler \u2013 Log & Continue",
      "type": "main",
      "index": 0
    }
  ]
]
},
"IF \u2013 Score > 100": {
  "main": [
    [
      {
        "node": "Enrich \u2013 Algolia Details (High Score)",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Label \u2013 Normal Score Story",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Enrich \u2013 Algolia Details (High Score)": {
  "main": [
    [
      {
        "node": "Merge \u2013 High Score Story",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Error Handler \u2013 Log & Continue",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Merge \u2013 High Score Story": {
  "main": [
    [
      {
        "node": "Build \u2013 Discord Digest Message",
        "type": "main",
        "index": 0
      }
    ]
  ]
}
]

```

```

},
"Label \u2013 Normal Score Story": {
  "main": [
    [
      {
        "node": "Build \u2013 Discord Digest Message",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Build \u2013 Discord Digest Message": {
  "main": [
    [
      {
        "node": "Discord \u2013 Post Digest",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Discord \u2013 Post Digest": {
  "main": [
    [],
    [
      {
        "node": "Error Handler \u2013 Log & Continue",
        "type": "main",
        "index": 0
      }
    ]
  ]
}
},
"pinData": {},
"settings": {
  "executionOrder": "v1"
},
"staticData": null,
"tags": [],
"triggerCount": 1,
"updatedAt": "2025-05-26T00:00:00.000Z",
"versionId": "1"
}

```


Section 3 — Bonus: Uptime Monitor Workflow

What it does

- Pings **demo.realworld.io** every **5 minutes** via Schedule trigger
- **Retry logic**: if non-200, waits and pings again before alerting — avoids false positives
- Posts a ■ downtime alert to Discord if outage is confirmed
- **Daily 08:00 summary** via a second cron trigger (0 8 * * *)
- Silent on success — logs to console only, no notification spam
- All HTTP nodes have onError routing to prevent silent failures

Workflow JSON (Bonus_UptimeMonitor_SrilathaPothana.json)

Import this JSON into n8n alongside Task 2. Uses the same Discord Webhook credential.

```
{
  "name": "Bonus \u2013 Uptime Monitor (demo.realworld.io)",
  "nodes": [
    {
      "parameters": {
        "rule": {
          "interval": [
            {
              "field": "minutes",
              "minutesInterval": 5
            }
          ]
        }
      },
      "id": "node-schedule-uptime",
      "name": "Schedule \u2013 Every 5 Minutes",
      "type": "n8n-nodes-base.scheduleTrigger",
      "typeVersion": 1.2,
      "position": [
        0,
        300
      ]
    },
    {
      "parameters": {
        "url": "https://demo.realworld.io",
        "options": {
          "timeout": 10000,
          "response": {
            "response": {
              "fullResponse": true
            }
          }
        }
      },
      "id": "node-ping",
      "name": "HTTP Ping \u2013 demo.realworld.io",
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.2,
      "position": [
```



```

    ]
  },
  {
    "parameters": {
      "url": "https://demo.realworld.io",
      "options": {
        "timeout": 10000,
        "response": {
          "response": {
            "fullResponse": true
          }
        }
      }
    },
    "id": "node-retry-ping",
    "name": "Retry Ping \u2013 Confirm Outage",
    "type": "n8n-nodes-base.httpRequest",
    "typeVersion": 4.2,
    "position": [
      1100,
      160
    ],
    "onError": "continueErrorOutput"
  },
  {
    "parameters": {
      "conditions": {
        "options": {
          "caseSensitive": true
        },
        "conditions": [
          {
            "id": "cond-confirmed",
            "leftValue": "={{ $json.statusCode }}",
            "rightValue": 200,
            "operator": {
              "type": "number",
              "operation": "notEquals"
            }
          }
        ]
      },
      "combinator": "and"
    },
    "options": {}
  },
  {
    "id": "node-if-confirmed",
    "name": "IF \u2013 Outage Confirmed?",
    "type": "n8n-nodes-base.if",
    "typeVersion": 2,
    "position": [
      1320,
      160
    ]
  },
  {
    "parameters": {
      "jsCode": "const d = $('Evaluate \u2013 Status & Response Time').first().json;\nconst ts = d.checkedAt;\nconst c",
    },
    "id": "node-build-alert",
    "name": "Build \u2013 Downtime Alert Message",
    "type": "n8n-nodes-base.code",
  }

```

```

    "typeVersion": 2,
    "position": [
      1540,
      80
    ]
  },
  {
    "parameters": {
      "method": "POST",
      "url": "={{ $credentials.discordWebhookUrl }}",
      "sendBody": true,
      "contentType": "json",
      "body": {
        "content": "={{ $json.message }}"
      },
      "options": {}
    },
    "id": "node-discord-alert",
    "name": "Discord \u2013 Send Alert",
    "type": "n8n-nodes-base.httpRequest",
    "typeVersion": 4.2,
    "position": [
      1760,
      80
    ],
    "credentials": {
      "httpCustomAuth": {
        "id": "cred-discord",
        "name": "Discord Webhook"
      }
    },
    "onError": "continueErrorOutput"
  },
  {
    "parameters": {
      "jsCode": "/* Site is up \u2013 log silently, no alert needed\nconst d = $input.first().json;\nconsole.log(`UPTIME: ${d} seconds`);"
    },
    "id": "node-ok",
    "name": "Log \u2013 Site is Up (No Alert)",
    "type": "n8n-nodes-base.code",
    "typeVersion": 2,
    "position": [
      880,
      440
    ]
  },
  {
    "parameters": {
      "jsCode": "/* Daily summary: aggregate uptime stats (placeholder \u2013 in production, read from DB/Google Sheet)"
    },
    "id": "node-daily-summary",
    "name": "Build \u2013 Daily Summary (08:00)",
    "type": "n8n-nodes-base.code",
    "typeVersion": 2,
    "position": [
      440,
      560
    ]
  },
  {
    "parameters": {

```

```

    "rule": {
      "interval": [
        {
          "field": "cronExpression",
          "expression": "0 8 * * *"
        }
      ]
    }
  },
  "id": "node-daily-trigger",
  "name": "Schedule \u2013 Daily 08:00 Summary",
  "type": "n8n-nodes-base.scheduleTrigger",
  "typeVersion": 1.2,
  "position": [
    220,
    560
  ]
},
{
  "parameters": {
    "method": "POST",
    "url": "={{ $credentials.discordWebhookUrl }}",
    "sendBody": true,
    "contentType": "json",
    "body": {
      "content": "={{ $json.message }}"
    },
    "options": {}
  },
  "id": "node-discord-summary",
  "name": "Discord \u2013 Send Daily Summary",
  "type": "n8n-nodes-base.httpRequest",
  "typeVersion": 4.2,
  "position": [
    660,
    560
  ],
  "credentials": {
    "httpCustomAuth": {
      "id": "cred-discord",
      "name": "Discord Webhook"
    }
  },
  "onError": "continueErrorOutput"
}
],
"connections": {
  "Schedule \u2013 Every 5 Minutes": {
    "main": [
      [
        {
          "node": "HTTP Ping \u2013 demo.realworld.io",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "HTTP Ping \u2013 demo.realworld.io": {
    "main": [
      [

```

```

        {
            "node": "Evaluate \u2013 Status & Response Time",
            "type": "main",
            "index": 0
        }
    ],
    [
        {
            "node": "Evaluate \u2013 Status & Response Time",
            "type": "main",
            "index": 0
        }
    ]
],
},
"Evaluate \u2013 Status & Response Time": {
    "main": [
        [
            {
                "node": "IF \u2013 Site Down?",
                "type": "main",
                "index": 0
            }
        ]
    ]
},
"IF \u2013 Site Down?": {
    "main": [
        [
            {
                "node": "Wait \u2013 Retry in 30s",
                "type": "main",
                "index": 0
            }
        ],
        [
            {
                "node": "Log \u2013 Site is Up (No Alert)",
                "type": "main",
                "index": 0
            }
        ]
    ]
},
"Wait \u2013 Retry in 30s": {
    "main": [
        [
            {
                "node": "Retry Ping \u2013 Confirm Outage",
                "type": "main",
                "index": 0
            }
        ]
    ]
},
"Retry Ping \u2013 Confirm Outage": {
    "main": [
        [
            {
                "node": "IF \u2013 Outage Confirmed?",
                "type": "main",

```

```

        "index": 0
    }
],
[
    {
        "node": "Build \u2013 Downtime Alert Message",
        "type": "main",
        "index": 0
    }
]
],
},
"IF \u2013 Outage Confirmed?": {
    "main": [
        [
            {
                "node": "Build \u2013 Downtime Alert Message",
                "type": "main",
                "index": 0
            }
        ],
        [
            {
                "node": "Log \u2013 Site is Up (No Alert)",
                "type": "main",
                "index": 0
            }
        ]
    ]
},
"Build \u2013 Downtime Alert Message": {
    "main": [
        [
            {
                "node": "Discord \u2013 Send Alert",
                "type": "main",
                "index": 0
            }
        ]
    ]
},
"Schedule \u2013 Daily 08:00 Summary": {
    "main": [
        [
            {
                "node": "Build \u2013 Daily Summary (08:00)",
                "type": "main",
                "index": 0
            }
        ]
    ]
},
"Build \u2013 Daily Summary (08:00)": {
    "main": [
        [
            {
                "node": "Discord \u2013 Send Daily Summary",
                "type": "main",
                "index": 0
            }
        ]
    ]
}

```

```
    ]
  }
},
"pinData": {},
"settings": {
  "executionOrder": "v1"
},
"staticData": null,
"tags": [],
"triggerCount": 2,
"updatedAt": "2025-05-26T00:00:00.000Z",
"versionId": "1"
}
```